

Synapse by Veridia — Arquitectura de Agentes y Estructura de Información

Versión: 1.1 | **Fecha:** 2026-03-19 | **Actualizado:** 2026-04-02 | **Uso:** Contexto de agente de desarrollo

Relacionado con: Synapse_Stack_v2.2.md, Synapse_SRS_v1.2.docx

1. Propósito de este documento

Este documento formaliza tres decisiones de arquitectura de IA para Synapse:

1. **La separación en dos agentes LLM especializados** — Entrevistador y Analista — y el rationale detrás de ella.
2. **La estructura de información de la sesión** — tres capas de memoria con naturalezas distintas y cómo se relacionan.
3. **La decisión de mantener la Knowledge Base como similitud vectorial en el MVP** y postergar su conversión a knowledge graph para la v2, con el rationale completo.

Estas decisiones tienen implicaciones directas en el diseño de prompts, el contrato de interfaces entre módulos, y la arquitectura del pipeline de IA.

2. Las tres capas de memoria de Synapse

Antes de describir los agentes, es necesario entender los tres sistemas de información que coexisten en una sesión activa. Son conceptualmente distintos y no deben mezclarse en implementación.

TRES CAPAS DE MEMORIA		
Knowledge Base (KB)	Knowledge Graph de Sesión	Transcript
Qué almacena	Qué almacena	Qué almacena
Conocimiento curado de Veridia: casos de éxito, benchmarks, frameworks	Mapa causal del problema del usuario actual: nodos tipados, aristas causales, métricas vinculadas	Historial literal de la conversación turno a turno
Cuándo se crea	Cuándo se crea	Cuándo se crea
Offline, por el equipo de Veridia	En tiempo real, turno a turno durante la sesión	En tiempo real, turno a turno
Cómo se consulta	Cómo se consulta	Cómo se consulta
Similitud vectorial (coseno)	Graph traversal + semántica (Graphiti hybrid)	Secuencial o comprimido
Dónde vive	Dónde vive	Dónde vive
pgvector en Supabase	Neo4j vía Graphiti	JSONB en Supabase (tabla sessions)
Pregunta que responde	Pregunta que responde	Pregunta que responde
"¿Qué casos conocidos se parecen a este problema?"	"¿Cuál es la estructura causal del problema diagnosticado hasta ahora?"	"¿Qué dijo exactamente el usuario en el turno N?"

Quién lo usa	Quién lo usa	Quién lo usa
Ambos agentes	Agente Analista	Agente
vía LlamaIndex	vía GraphEngine	Entrevistador
	CausalAuditor	vía LangGraph
	Canvas	

El transcript es la materia prima. El knowledge graph es la interpretación estructurada de ese transcript. La KB es el contexto externo que enriquece esa interpretación con experiencia previa de Veridia.

3. Arquitectura de dos agentes — Entrevistador y Analista

3.1 El problema de un agente único

Un agente único que intenta simultáneamente generar la próxima pregunta y actualizar el knowledge graph enfrenta un conflicto de objetivos inherente.

Modo conversacional (lo que necesita el Entrevistador):

- Mantener coherencia emocional con lo que dijo el usuario
- Aplicar fricción positiva sin parecer interrogatorio
- Cubrir OMs sin que la entrevista parezca un formulario
- Adaptar el tono al perfil del usuario (C-Level vs Consultor)

Modo analítico (lo que necesita el Analista):

- Clasificar con precisión tipos semánticos de nodos (Síntoma vs Proceso vs Solución)
- Determinar si una relación es causal o correlacional
- Identificar gaps en el grafo que requieren más exploración
- Detectar patrones emergentes comparando con la KB

Estos dos modos compiten cuando coexisten en el mismo prompt. Un modelo optimizado para ser conversacional tiende a suavizar las inferencias estructurales para mantener coherencia narrativa. Un modelo optimizado para ser estructuralmente preciso tiende a generar preguntas que suenan técnicas y pierden la calidez necesaria para que el usuario se abra.

En sesiones largas (>20 turnos) este conflicto se degrada de forma observable: el agente pierde el hilo conversacional, o empieza a generar nodos genéricos para "terminar rápido" con el análisis.

3.2 La solución: separación de responsabilidades cognitivas

AGENTE ENTREVISTADOR	AGENTE ANALISTA
Responsabilidad única:	Responsabilidad única:
"¿Qué pregunto ahora?"	"¿Qué estructura emerge de esta respuesta?"

Esta separación no es sobre el formato del conocimiento (vectorial vs grafo) — es sobre que los dos agentes tienen **modos cognitivos genuinamente distintos** que se sabotean mutuamente cuando conviven en un solo modelo.

La separación es válida independientemente de si la KB es vectorial o un knowledge graph, y se mantiene válida si en el futuro ambas estructuras de información son grafos. La razón es que cuando la KB sea también un grafo, el Analista se vuelve más capaz (puede comparar subgrafos en lugar de vectores), pero el Entrevistador sigue necesitando decidir qué hacer con esa riqueza en el contexto conversacional del momento. Esa decisión editorial es su trabajo y no desaparece.

4. Contexto de cada agente

4.1 Agente Entrevistador

AGENTE ENTREVISTADOR	
LO QUE SÍ NECESITA	LO QUE NO NECESITA
Transcript completo de la conversación	Nodos y aristas específicos del grafo
Estado de los 6 IMOs (% de cobertura de cada uno)	Tipos semánticos de cada arista
Resumen del grafo actual (áreas mapeadas, no detalles)	Si un nodo es hipótesis o verificado
Fragmentos de KB relevantes al hilo conversacional actual	Las métricas validadas en detalle
exploration_gaps del Analista (en lenguaje natural)	Los IDs internos de los nodos en Neo4j
emerging_hypotheses del Analista (en lenguaje natural)	
Perfil del usuario (C-Level vs Consultor)	

Pregunta que responde:

"¿Qué debo preguntarle al usuario a continuación para avanzar el diagnóstico de forma coherente con la conversación y los gaps identificados por el Analista?"

Modelo: Claude claude-sonnet-4-6 (Bedrock) o Gemini 2.0 Flash (Vertex AI) según el motor del workspace

4.2 Agente Analista

AGENTE ANALISTA

LO QUE SÍ NECESITA	LO QUE NO NECESITA
La respuesta validada del usuario (turno actual)	El transcript completo de la conversación
La pregunta que generó esa respuesta (contexto inmediato)	Los IMO scores
El grafo completo actual (nodos, aristas, tipos, estados, métricas vinculadas)	El historial de preguntas anteriores
Fragmentos de KB relevantes para comparar patrones causales	El perfil de usuario (C-Level vs Consultor)
Las métricas ya validadas y sus valores	

Pregunta que responde:

"¿Qué estructura causal nueva emerge de esta respuesta?
¿Qué nodos añado, actualizo, o promuevo de hipótesis a verificado? ¿Qué gaps quedan sin explorar?"

Modelo: Claude claude-sonnet-4-6 (Bedrock) o Gemini 2.0 Flash (Vertex AI)
según el motor del workspace

5. El contrato de información entre agentes — AnalystOutput

El Analista no le pasa el grafo completo al Entrevistador — le pasa una **vista editorial** del grafo en lenguaje natural. Esta distinción es fundamental.

El Entrevistador no necesita saber que existe un nodo `Proceso` con `id: node_47` en Neo4j. Necesita saber que "el área de procesos de RRHH está poco explorada y hay una hipótesis

emergente que vale la pena profundizar". Eso es exactamente lo que un consultor humano necesitaría saber para conducir la siguiente parte de la entrevista.

```

class AnalystOutput(BaseModel):
    """
    Output del Agente Analista tras procesar un turno validado.
    Se divide en dos partes: lo que va al GraphEngine/Canvas,
    y lo que va al Agente Entrevistador.
    """

    # — Para GraphEngine y Canvas —————
    graph_delta: GraphDelta
    # Nodos nuevos, nodos actualizados, aristas nuevas,
    # aristas actualizadas, nodos promovidos hipótesis-verificado.
    # El GraphEngine persiste esto en Neo4j y el Canvas lo renderiza.

    # — Para el Agente Entrevistador —————
    exploration_gaps: list[str]
    # Áreas del problema que no han sido exploradas aún,
    # en lenguaje natural.
    # Ejemplo:
    #   "No hay métricas sobre el proceso de contratación
    #   pese a que fue mencionado como posible causa raíz"
    #   "El área de costos operativos no ha sido cuantificada"

    emerging_hypotheses: list[str]
    # Patrones causales que están tomando forma pero sin confirmar.
    # Ejemplo:
    #   "La rotación podría estar causada por onboarding deficiente,
    #   no por compensación como sugirió el usuario inicialmente"
    #   "El nodo 'supervisión' aparece como posible causa raíz
    #   de tres síntomas distintos"

    omi_coverage_update: dict[str, float]
    # OMIs que avanzaron con este turno y su nuevo score estimado.
    # Ejemplo: {"IMO-2": 0.7, "IMO-4": 0.3}
    # El Entrevistador usa esto para decidir qué OMIs priorizar.

    kb_match: KBMatch | None
    # Si el retrieval RAG encontró un caso similar en este turno.
    # Se pasa al Entrevistador para que decida si mencionarlo
    # o usarlo como referencia implícita en la siguiente pregunta.

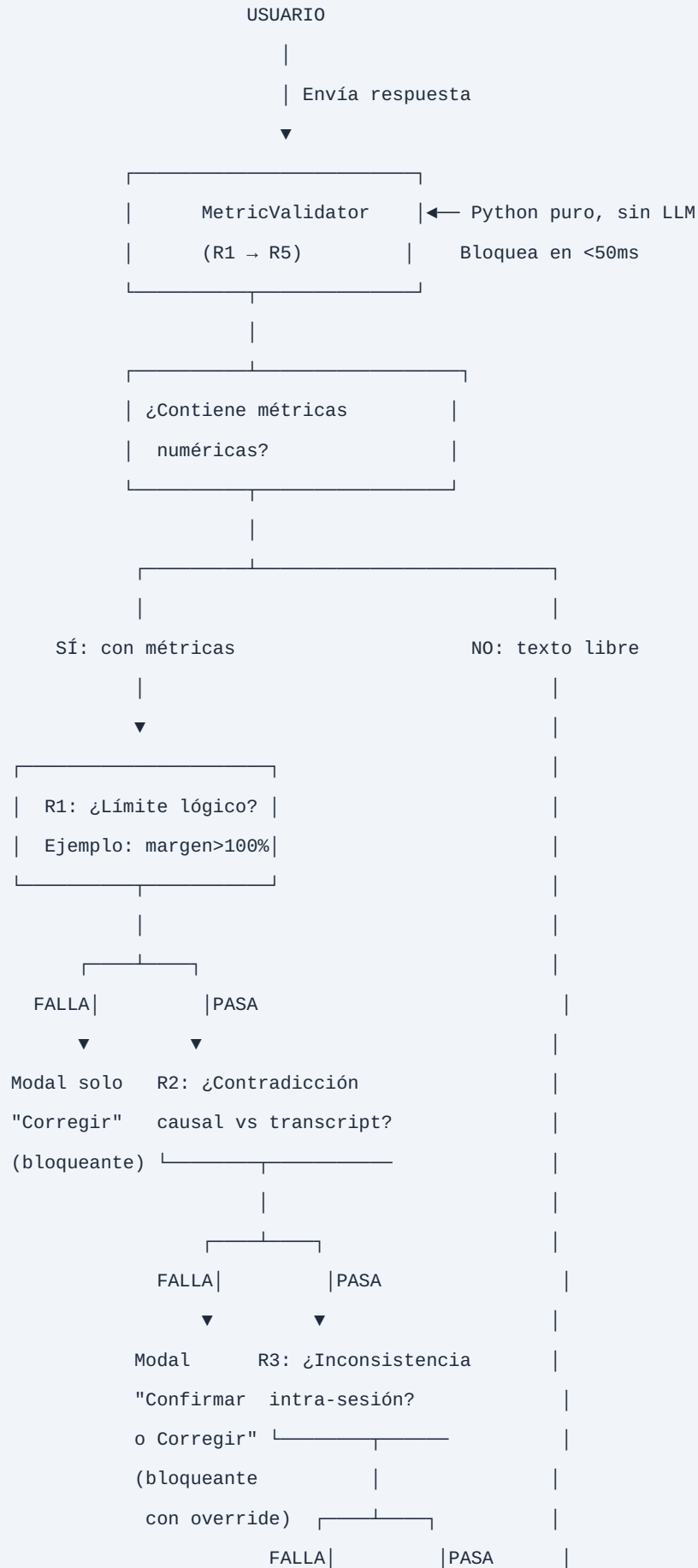
```

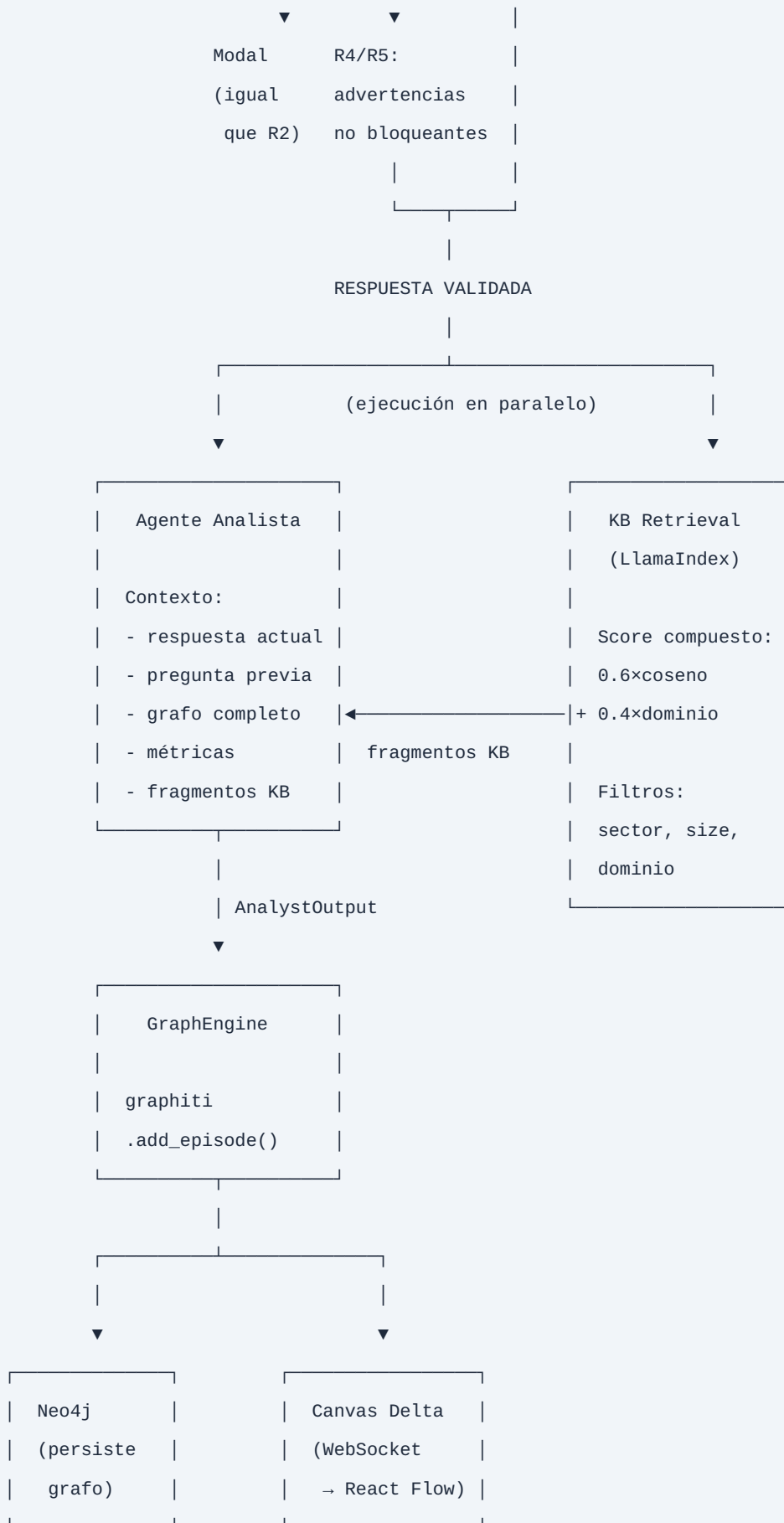


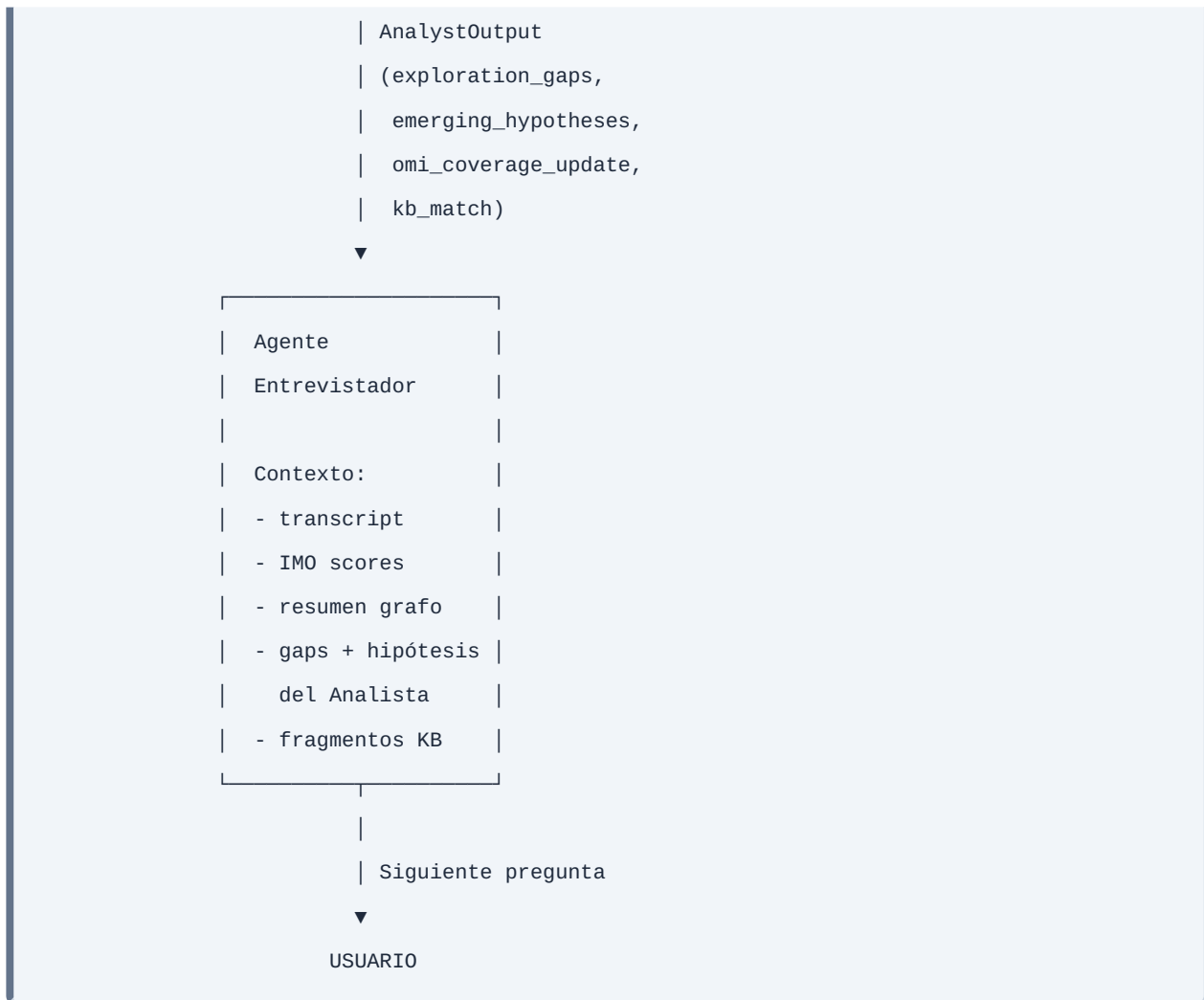
```
# Nota: el "Similar Case Detected" visible en el canvas  
# se renderiza desde aquí, no desde el Entrevistador.
```

6. El ciclo completo de un turno

Este es el flujo de información para cada turno de la entrevista después de que el usuario envía una respuesta. Representa el estado del sistema durante la fase `interviewing` (UC-02).







6.1 Notas sobre el ciclo

Paralelismo: El Agente Analista y el retrieval de KB corren en paralelo. El Analista no espera a que el retrieval termine — recibe los fragmentos de KB como input adicional cuando están disponibles, o trabaja sin ellos si el retrieval tarda más de un umbral configurable.

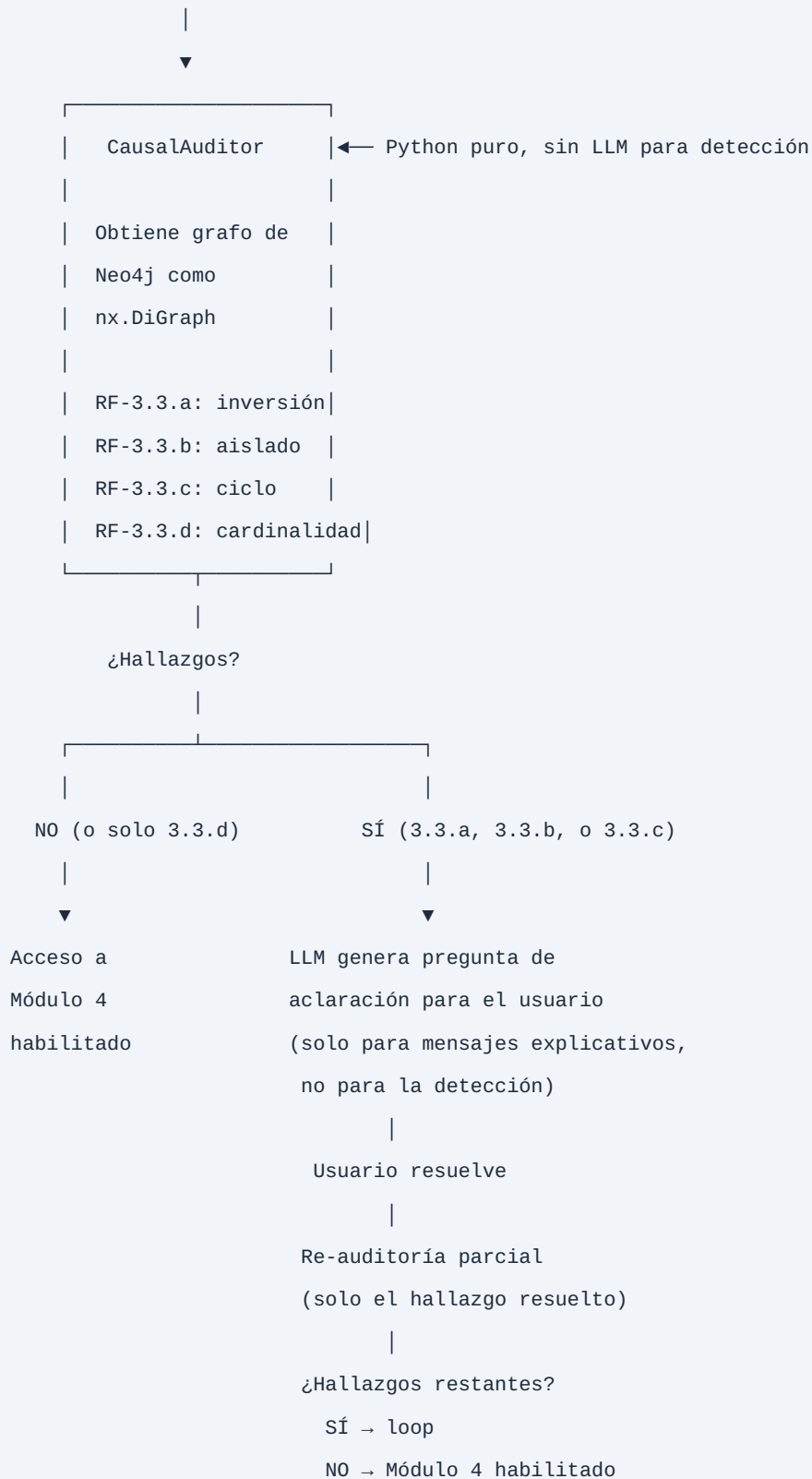
No bloqueante para el canvas: El usuario recibe la siguiente pregunta del Entrevistador mientras el Analista procesa el delta del grafo en background. El canvas se actualiza unos segundos después de que aparece la siguiente pregunta, no antes. Este comportamiento asíncrono es intencional y está especificado en el SRS.

Snapshot automático: Después de que el GraphEngine persiste el delta, el `SnapshotManager` guarda un snapshot inmutable con el trigger `chat_response_confirmed`. Este proceso no está en el camino crítico — ocurre en background después de que el turno está completo.

7. Flujo de información en la auditoría causal (UC-03)

Cuando el usuario intenta avanzar a Módulo 4, antes de habilitar el acceso se ejecuta la auditoría causal. Este flujo es distinto al ciclo de turno.

Usuario solicita avanzar a Módulo 4



8. Estructura de información de la KB en el MVP

8.1 Decisión: KB como similitud vectorial en el MVP

La Knowledge Base de Veridia se implementa como un índice vectorial en pgvector + LlamaIndex. Los documentos se representan como embeddings de `text-embedding-004` (Gemini) y el retrieval usa similitud coseno + filtros de metadata.

El formato de representación en el MVP es vectorial, no estructural.

8.2 Rationale

La KB vectorial es suficiente para el MVP. La similitud semántica permite encontrar casos parecidos con precisión razonable para el volumen inicial de la KB. La complejidad adicional de un knowledge graph para la KB no está justificada cuando hay pocos casos y la prioridad es validar el producto.

El pipeline de ingesta es radicalmente más simple. Generar un embedding de un documento tarda milisegundos. Construir un knowledge graph desde ese documento requiere extracción de entidades por LLM, resolución de duplicados, inferencia de relaciones, y persistencia en Neo4j — es un pipeline de procesamiento offline no trivial.

Los 150 horas del MVP son mejor utilizadas en el producto. El Agente Analista, el canvas interactivo, y el pipeline RAG básico son las piezas que validan la propuesta de valor. La KB como grafo es una optimización que mejora la calidad del producto en iteraciones posteriores.

El fallo es silencioso con vectores. Un documento mal redactado tiene un embedding menos preciso — el retrieval devuelve resultados menos relevantes. Con un knowledge graph, un documento mal redactado puede generar nodos duplicados, aristas incorrectas, o entidades mal tipadas — el fallo es más disruptivo y difícil de diagnosticar.

8.3 Limitación conocida de la KB vectorial

Con KB vectorial, cuando el Agente Analista busca casos similares, la similitud es **semántica** (los textos se parecen) pero no **estructural** (los patrones causales son los mismos). El sistema puede encontrar que dos casos hablan de "rotación de personal" y "onboarding" sin detectar que en uno el onboarding es causa raíz y en el otro es consecuencia de algo más profundo.

Esta limitación es aceptable en el MVP porque:

- El Agente Analista usa la KB como contexto de enriquecimiento, no como fuente de verdad estructural.
- El score compuesto (60% coseno + 40% dominio) ya incorpora un filtro estructural básico vía metadata.
- La precisión del retrieval mejorará con el volumen de la KB, independientemente del formato.

9. Decisión postergada para v2: KB como Knowledge Graph

9.1 La visión

En la v2, la KB de Veridia se convierte en un knowledge graph en Neo4j usando el mismo schema de nodos y aristas que el grafo de sesión. Ambas estructuras hablan el mismo lenguaje representacional.

```
Neo4j AuraDB (v2)
├─ group: veridia_kb          ← construido offline por Veridia
|   Nodos: Síntoma, Proceso,      actualizado cuando se añaden casos
|       Solución, Resultado
|   Aristas: causales, temporales
|
└─ group: session_{id}       ← construido en tiempo real
    Nodos: mismos tipos
    Aristas: mismos tipos
```

9.2 Qué cambia cuando la KB es un KG

El retrieval se vuelve estructural además de semántico. El Analista puede buscar subgrafos en la KB con topología causal similar al grafo de sesión en construcción — no "este texto se parece" sino "esta cadena Síntoma → Proceso → Resultado tiene el mismo patrón causal".

El output del Analista se enriquece. El contrato `AnalystOutput` puede incluir un `kb_pattern_match` con evidencia estructural:

```
# MVP (KB vectorial)
kb_match: KBMatch | None
# "Este caso habla de temas similares al problema del usuario"

# v2 (KB como KG)
kb_pattern_match: KBPatternMatch | None
# "Esta cadena causal tiene la misma topología que el caso X,
# con la diferencia de que en ese caso el problema raíz estaba
# en supervisión, no en el proceso formal de inducción.
# Pregunta sugerida: ¿cómo describirías la relación entre
# los supervisores y los nuevos ingresos?"
```

La clasificación de horizonte se vuelve estructural. En lugar de que el LLM clasifique H1/H2/H3 basándose en texto, compara la topología del grafo de sesión con los grafos de casos conocidos de cada horizonte.

9.3 Qué NO cambia con la KB como KG

La separación Entrevistador / Analista sigue siendo válida. La razón de la separación es la distinción de modos cognitivos, no el formato del conocimiento. Cuando la KB sea un KG, el Analista se vuelve más capaz, pero el Entrevistador sigue necesitando decidir qué hacer con esa riqueza en el contexto conversacional del momento.

9.4 Prerequisitos para la migración a v2

Antes de implementar la KB como KG, deben cumplirse:

1. **Schema estabilizado.** Los tipos de nodos y aristas del grafo de sesión deben estar consolidados y validados con sesiones reales. Migrar la KB a un schema que luego cambia es trabajo doble.
2. **Pipeline de ingesta definido.** El proceso editorial de Veridia (quién cura los casos, en qué formato los provee, quién valida la extracción de entidades) debe estar documentado antes de automatizarlo.
3. **Volumen mínimo de casos.** Con menos de ~30 casos en la KB, la búsqueda por similitud estructural no aporta sobre la vectorial. El beneficio escala con el volumen.
4. **OI-01 del SRS resuelto.** El Open Issue sobre el proceso editorial de la KB es bloqueante para la v2.

10. Decisiones de implementación derivadas

Las siguientes decisiones de implementación se derivan directamente de la arquitectura de agentes definida en este documento. Son constraints, no sugerencias.

Decisión	Descripción
D-IA-01	El <code>SessionController</code> orquesta los dos agentes. Ningún otro módulo llama directamente a un LLM.
D-IA-02	El Agente Analista corre de forma no bloqueante. El Entrevistador genera la siguiente pregunta mientras el Analista procesa en background.
D-IA-03	El Entrevistador recibe el output del Analista en lenguaje natural (<code>exploration_gaps</code> , <code>emerging_hypotheses</code>), nunca estructuras de grafo (nodos, aristas, IDs).
D-IA-04	El <code>MetricValidator</code> y el <code>CausalAuditor</code> son Python puro. No se convierten en agentes LLM. El LLM solo genera los mensajes explicativos, nunca toma las decisiones de bloqueo.
D-IA-05	El Agente Analista no tiene acceso al transcript completo. Solo recibe la respuesta validada del turno actual y la pregunta que la generó.
D-IA-06	El Agente Entrevistador no tiene acceso al grafo detallado. Solo recibe el <code>GraphSummary</code> generado por <code>GraphEngine.get_graph_summary()</code> .
D-IA-07	El retrieval de KB corre en paralelo con el Agente Analista. El Analista no espera bloqueado por el retrieval.
D-IA-08	Un solo índice vectorial para toda la KB. El retrieval para "Similar Case Detected" (UC-02) y para el Módulo 4 (RF27) usa el mismo índice con filtros distintos.

11. Estado de implementación de agentes (actualizado 2026-04-02)

Agente / Módulo	Estado	Notas de implementación
AgentEntrevistador (interviewer.py)	✓ Funcional	max_tokens=8192 . Streaming vía WebSocket. Recibe transcript comprimido + IMO scores + GraphSummary + exploration_gaps.
AgentAnalista (analyst.py)	✓ Funcional	max_tokens=4096 . Devuelve AnalystOutput en JSON estructurado. Corre en paralelo con KB Retrieval.
SessionController (session_controller.py)	✓ Funcional	LangGraph con estados: idle → interviewing → exploration_complete → audit_pending → audit_passed → proposals_ready .
InterviewController	✓ Funcional	IMO scoring (6 IMOs), coverage tracking, detección de exploration_complete .
ContextManager (context_manager.py)	✓ Funcional	Compresión de transcript a partir de 40 turnos. Separación estricta de contexto Entrevistador/ Analista.
MetricValidator	✓ Funcional	5 reglas semánticas deterministas (R1-R5). Sin LLM.
CausalAuditor	✗ Stub	Pendiente Semana 5 (Módulo 3).
GraphEngine	✗ Stub	Pendiente Semana 4 (Módulo 2).
KBRetriever	✓ Funcional	Score compuesto $0.6 \times \text{coseno} + 0.4 \times \text{dominio}$. Filtros por sector, size, dominio.

Configuración de límites de tokens

Por defecto en producción:

Provider/Agente	<code>max_tokens</code> configurado	Razón
<code>GeminiVertexProvider</code>	8192	Prev. truncamiento en respuestas largas del Entrevistador
<code>ClaudeBedrockProvider</code>	8192	Ídem, para workspaces con plan Professional/Enterprise
<code>AgentEntrevistador</code>	8192	Preguntas + síntesis de razonamiento pueden ser extensas
<code>AgentAnalista</code>	4096	JSON estructurado es compacto; 4096 es más que suficiente

Nota: Aumentar el límite no aumenta el costo si la respuesta real es más corta — los tokens se cobran por los generados, no por el límite declarado. El límite previene cortes bruscos en respuestas largas.